

Quem recebeu o dinheiro da cota parlamentar?

Um arquivo público, 209 mil linhas, ninguém limpou nada. Vamos responder duas vezes: primeiro errado, depois certo. A diferença entre as duas respostas é a aula inteira.

O DADO

Uma nota fiscal por linha.

CEAP — Cota para o Exercício da Atividade Parlamentar. Cada deputado tem uma verba mensal, gasta, e presta contas. A Câmara publica o arquivo inteiro.

Ano-2025.csv 74,5 MB 32 colunas 209.066 lançamentos domínio público.

A RESPOSTA INGÊNUA

R\$ 16,4 mi

TAM LINHAS AÉREAS

A RESPOSTA INGÊNUA

R\$ 16,4 mi

ERRADO

TAM LINHAS AÉREAS

Uma **GROUP BY** honesta, num dado honesto, que produz uma resposta falsa.

POR QUE A RESPOSTA É FALSA

Companhia aérea não é fornecedora de deputado.

É o **SIGEPA**, o sistema de passagens da própria Câmara. O bilhete sai por lá e a companhia entra na planilha. O dado não mentiu — **a pergunta é que foi mal feita.**

O dado sujo não avisa que está sujo. Ele só responde.

Três defeitos neste arquivo. Nenhum é bug. Todos são o mundo real chegando sem aviso.

DEFEITO 1 DE 3

Oito mil linhas com valor negativo.

8.017

ESTORNOS – DEVOLUÇÕES DE DESPESA

Oito mil linhas com valor negativo.

8.017

ESTORNOS – DEVOLUÇÕES DE DESPESA

Não apague. Um estorno é informação: alguém devolveu dinheiro. Ele vira uma coluna `eh_estorno`, e cada pergunta decide se quer contar com ele ou sem.

DEFEITO 2 DE 3

Trinta e sete mil linhas sem CNPJ.

37.934

SEM DOCUMENTO DO FORNECEDOR

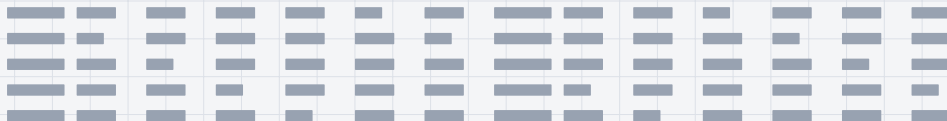
Trinta e sete mil linhas sem CNPJ.

37.934

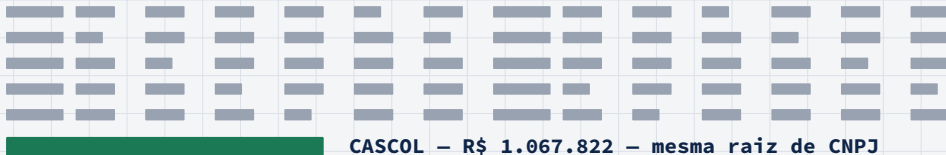
SEM DOCUMENTO DO FORNECEDOR

Sem CNPJ não existe **chave natural**: você não consegue ligar essa linha a nenhuma informação de fora do arquivo. Vira a coluna `via_sigepa`.

Setenta fornecedores que são um só.



Setenta fornecedores que são um só.



Nenhum pedaço era grande o bastante para aparecer sozinho. Juntos, mudam o pódio.

O CONCERTO PRECISA DE UM LUGAR PARA MORAR

Ano-2025.csv

74,5 MB • 32 COLUNAS

CAMARA.LEG.BR

O CONCERTO PRECISA DE UM LUGAR PARA MORAR



O CONCERTO PRECISA DE UM LUGAR PARA MORAR



TELA 1 • O QUE UMA TABELA DELTA É POR DENTRO

```
mc ls - container minio - s3a://lake
```

```
$ mc ls --recursive local/lake/
```

```
bronze/ -- _delta_log/00000000000000000000.json 2.1 KiB
```

```
bronze/ -- part-00000-....snappy.parquet 17 MiB
```

```
silver/ -- _delta_log/00000000000000000000.json 3.4 KiB
```

```
silver/ -- part-00000-....snappy.parquet 6.1 MiB
```

```
gold/ -- _delta_log/00000000000000000000.json 1.7 KiB
```

```
gold/ -- part-00000-....snappy.parquet 1.2 MiB
```

TELA 1 • O QUE UMA TABELA DELTA É POR DENTRO

```
mc ls - container minio - s3a://lake
```

```
$ mc ls --recursive local/lake/
```

```
bronze/ -- _delta_log/00000000000000000000.json 2.1 KiB
```

```
bronze/ -- part-00000-....snappy.parquet 17 MiB
```

```
silver/ -- _delta_log/00000000000000000000.json 3.4 KiB
```

```
silver/ -- part-00000-....snappy.parquet 6.1 MiB
```

```
gold/ -- _delta_log/00000000000000000000.json 1.7 KiB
```

```
gold/ -- part-00000-....snappy.parquet 1.2 MiB
```

Uma pasta de Parquet mais **um log de transações**. É esse log que dá à pasta as coisas que você espera de uma tabela: escrita atômica, leitura consistente e histórico.

POR QUE TRÊS CAMADAS E NÃO UMA

CAMADA 1

bronze

Como a Câmara publicou. Tudo texto. Nada consertado.

CAMADA 2

silver

Tipos certos, defeitos marcados em colunas. Nada apagado.

CAMADA 3

gold

Uma tabela por pergunta. 19.441 empresas, uma linha cada.

POR QUE TRÊS CAMADAS E NÃO UMA

CAMADA 1

bronze

Como a Câmara publicou. Tudo texto. Nada consertado.

CAMADA 2

silver

Tipos certos, defeitos marcados em colunas. Nada apagado.

CAMADA 3

gold

Uma tabela por pergunta. 19.441 empresas, uma linha cada.

O bronze não é desperdício: é o seguro. **Sua regra de limpeza vai estar errada uma hora** — e aí o bronze é o único jeito de refazer sem pedir o arquivo de novo.

TELA 2 • A MESMA PERGUNTA, NAS DUAS PONTAS

spark-sql – container spark-master

```
spark-sql> SELECT txtFornecedor, sum(vlrDocumento) t  
              FROM delta.`s3a://lake/bronze` GROUP BY 1 ORDER BY 2 DESC LIMIT 3;
```

```
TAM 16.402.463,26  GOL 6.052.550,88  AZUL 4.696.245,04
```

```
spark-sql> SELECT fornecedor, total  
              FROM delta.`s3a://lake/gold` ORDER BY total DESC LIMIT 3;
```

```
FACEBOOK 3.251.951,87  PANTANAL 3.227.314,77  TELEFÔNICA 1.591.715,84
```

TELA 2 • A MESMA PERGUNTA, NAS DUAS PONTAS

```
spark-sql – container spark-master

spark-sql> SELECT txtFornecedor, sum(vlrDocumento) t
              FROM delta.`s3a://lake/bronze` GROUP BY 1 ORDER BY 2 DESC LIMIT 3;
TAM 16.402.463,26  GOL 6.052.550,88  AZUL 4.696.245,04

spark-sql> SELECT fornecedor, total
              FROM delta.`s3a://lake/gold` ORDER BY total DESC LIMIT 3;
FACEBOOK 3.251.951,87  PANTANAL 3.227.314,77  TELEFÔNICA 1.591.715,84
```

Mesmo arquivo. Mesma pergunta. **Nenhum nome se repete entre as duas listas.**

A RESPOSTA, DEPOIS DAS TRÊS CAMADAS

R\$ 3,25 mi

FACEBOOK SERVIÇOS ONLINE DO BRASIL

A RESPOSTA, DEPOIS DAS TRÊS CAMADAS

R\$ 3,25 mi

FACEBOOK SERVIÇOS ONLINE DO BRASIL

Não foi o dado que mudou. **Foi o cuidado.**

Dois motores, o mesmo arquivo, contagens diferentes.

DUCKDB

209.066

`multiLine` é **true** por padrão

APACHE SPARK

209.079

`multiLine` é **false** por padrão

Dois motores, o mesmo arquivo, contagens diferentes.

DUCKDB

209.066

`multiLine` é **true** por padrão

APACHE SPARK

209.079

`multiLine` é **false** por padrão

13 registros têm quebra de linha **dentro** de um campo entre aspas. Nenhum motor está com defeito: os padrões é que são diferentes. **“O dado está certo” não é propriedade do arquivo — é do arquivo mais de quem lê.**

spark-sql – DESCRIBE HISTORY

VERSÃO	OPERAÇÃO	LINHAS	O QUE ERA
0	WRITE	209.079	sem multiLine – 13 linhas partidas
1	WRITE	209.066	com multiLine – correto

```
spark-sql> SELECT count(*) FROM delta.`s3a://lake/bronze` VERSION AS OF 0;  
209079
```

TELA 3 • O ERRO CONTINUA CONSULTÁVEL

spark-sql – DESCRIBE HISTORY

VERSÃO	OPERAÇÃO	LINHAS	O QUE ERA
0	WRITE	209.079	sem multiLine – 13 linhas partidas
1	WRITE	209.066	com multiLine – correto

```
spark-sql> SELECT count(*) FROM delta.`s3a://lake/bronze` VERSION AS OF 0;  
209079
```

O erro não foi apagado — **ele virou uma versão**. Errou a limpeza? Volta, compara, conserta. E ninguém precisa baixar 74 MB da Câmara de novo.

PROCESSAMENTO • CONTAINERS spark

spark-master

REPARTE O TRABALHO

worker 1

2 CORES • 2 GB

worker 2

2 CORES • 2 GB

worker 3

SOBE AO VIVO



s3a:// — A MESMA API DO AMAZON S3

TROCA O ENDPOINT E O MESMO CÓDIGO RODA NA AWS

ARMAZENAMENTO • CONTAINER minio

MinIO

BUCKET lake

bronze

209.066 LINHAS

silver

DEFEITOS MARCADOS

gold

19.441 EMPRESAS

o Spark não guarda o dado. derrube os três containers dele e o lake continua inteiro.

TELA 4 • MAIS MÁQUINA, SEM TOCAR NO DADO

● ● ● docker compose – ao vivo

```
$ docker compose up -d --scale spark-worker=3
Container semuni-spark-worker-3 Started 0.9s

$ curl -s localhost:8080 | grep -c "ALIVE"
3
```

TELA 4 • MAIS MÁQUINA, SEM TOCAR NO DADO

```
docker compose --ao vivo

$ docker compose up -d --scale spark-worker=3
Container semuni-spark-worker-3 Started 0.9s

$ curl -s localhost:8080 | grep -c "ALIVE"
3
```

Uma linha, um worker novo, **e o dado não se mexeu**. Isso só é possível porque o dado nunca esteve dentro do Spark. É essa separação que faz o resto funcionar.

O MESMO DESENHO, COM O NOME QUE ELE TEM NA NUVEM

PROCESSAMENTO

Apache Spark

NO SEU DOCKER



EMR • Dataproc • Databricks • Synapse

O MESMO APACHE SPARK, OPERADO POR OUTRA PESSOA

ARMAZENAMENTO

MinIO

NO SEU DOCKER



Amazon S3 • Google Cloud Storage • Azure Blob

A MESMA API. VOCÊ TROCA UMA LINHA DE CONFIGURAÇÃO.

Delta Lake é o mesmo formato dos dois lados. Ele é aberto, e não muda quando você paga.

NO SEU DOCKER

Bom quando...

- você está aprendendo e quer quebrar sem medo
- o dado cabe confortavelmente numa máquina
- o ciclo curto importa: subiu, errou, refez em minutos
- o dado não pode sair da sua rede
- o custo precisa ser previsível

NA NUVEM

Bom quando...

- o dado não cabe mais numa máquina só
- você precisa de 200 máquinas por 20 minutos
- durabilidade e backup viram contrato, não tarefa sua
- o time é pequeno e o plantão não pode ser você
- outras equipes precisam do mesmo dado, com governança

NO SEU DOCKER

Bom quando...

- você está aprendendo e quer quebrar sem medo
- o dado cabe confortavelmente numa máquina
- o ciclo curto importa: subiu, errou, refez em minutos
- o dado não pode sair da sua rede
- o custo precisa ser previsível

NA NUVEM

Bom quando...

- o dado não cabe mais numa máquina só
- você precisa de 200 máquinas por 20 minutos
- durabilidade e backup viram contrato, não tarefa sua
- o time é pequeno e o plantão não pode ser você
- outras equipes precisam do mesmo dado, com governança

PARA LEVAR PARA CASA

O caminho até a nuvem começa num `docker compose up`.

Tudo que você viu hoje é software aberto: MinIO, Apache Spark, Delta Lake, Docker — e um CSV público da Câmara. Roda no seu laptop hoje, e **o mesmo código sobe na nuvem no dia em que o dado crescer.**

`make subir` • `make pipeline`